

Course Outline: Validating and Serializing API Data with Pydantic

Title: Validating and Serializing API Data with Pydantic **Skill:** __ — API Data Validation **Learnpack:** + Validando y serializando datos de APIs con Pydantic **File:** s__-w#d#-validando-y-serializando-datos-de-apis-con **Prerequisite:** Introduction to FastAPI **Target Audience:** Beginner developers building REST APIs with FastAPI **Total Lessons:** 11 **Format:** Mixed (18% hands-on)

Course Description

Every API receives data from the outside world and sends data back — but external data is untrustworthy by nature. Without validation, a missing field or a wrong type can crash your server or corrupt your database silently. Pydantic is the library that solves this problem: it lets you declare the shape of your data as Python classes, and automatically validates, parses, and serializes that data at runtime. FastAPI uses Pydantic natively, so once you know how to build Pydantic models, you get request validation and response serialization almost for free. This course teaches students to define data schemas, validate what comes into their endpoints, and control exactly what goes out.

Course Philosophy

This course emphasizes:

- Understanding the problem before the solution — data validation is a real-world necessity, not academic overhead
 - Learning the building block (BaseModel) before applying it, so the FastAPI integration feels natural rather than magical
 - Seeing both directions of an endpoint — incoming data and outgoing data — as two sides of the same Pydantic model pattern
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Pydantic	4
02 - Pydantic in FastAPI	4
03 - Assessment	1
04 - Outro	1
Total	11

00.0 Welcome to Pydantic

Learning Objectives:

- Understand what problem this course solves and why data validation matters in APIs
- Preview the two core skills students will develop: modeling data and using models in endpoints

Content Outline:

- The hidden danger of unvalidated API data — wrong types, missing fields, unexpected shapes
- How Pydantic fits into the FastAPI ecosystem
- What students will be able to do by the end: define schemas, validate inputs, shape responses

Transitions: Section 01 starts by introducing Pydantic itself — what it is, how it works, and how to define your first model.

Key Principles:

- External data cannot be trusted — validation is not optional in production APIs
- Pydantic moves validation from runtime guesswork to explicit, declarative schemas

Examples:

- A FastAPI endpoint that receives a JSON body with no validation — what happens when a field is missing or the wrong type
 - The same endpoint after adding a Pydantic model — automatic 422 error, clear error message, no extra code
-

01.0 Pydantic

Learning Objectives:

- Understand what Pydantic is and where it sits in the Python ecosystem
- Articulate the core problem Pydantic solves and why it matters for API development
- Preview the sub-lessons in this section: how Pydantic works and how to build models

Content Outline:

- The data validation problem in APIs — why raw dicts and manual checks fail at scale
- What Pydantic is: a data validation and settings management library powered by Python type annotations
- Where Pydantic fits: used by FastAPI, widely adopted in production Python backends
- What this section covers: mechanics first, then building your own models

Transitions: Before writing a Pydantic model yourself, you need to understand what happens under the hood when Pydantic validates data. That is what lesson 01.1 covers.

Key Principles:

- Pydantic uses Python type hints as the source of truth — the same annotations you write for documentation and IDE support now enforce runtime behavior
- Validation happens at object creation — invalid data raises an error immediately, not silently

Examples:

- A plain Python dict vs a Pydantic model representing a `User` with `name: str` and `age: int` — what happens when you pass `age: "twenty"`
- Pydantic's error output: structured, readable, and precise about what went wrong

Anti-patterns woven in:

- Using `dict` everywhere and validating manually with `isinstance()` — fragile, verbose, and easy to forget; Pydantic centralizes this in one place

01.1 How Pydantic Works

Learning Objectives:

- Explain how Pydantic parses and coerces input data using type annotations
- Describe what happens when validation fails and what a `ValidationError` contains
- Distinguish between strict mode and lenient coercion

Content Outline:

- `BaseModel`: the class all Pydantic models inherit from
- How Pydantic reads field annotations at class definition time and builds a schema
- Type coercion: Pydantic tries to convert `"42"` to `int` before raising an error — why this is useful and when it can surprise you
- `ValidationError`: when it is raised, what it contains, how to read its `.errors()` output
- Strict mode: opting out of coercion when you need exact types

Transitions: Now that you understand how Pydantic validates data internally, lesson 01.2 shows you how to define your own models with fields, types, defaults, and optional values.

Key Principles:

- Pydantic coerces by default — `"42"` becomes `42` for an `int` field — but strict mode disables this
- `ValidationError` is your friend: it tells you exactly which field failed, what value was received, and what was expected
- Validation happens at instantiation — `MyModel(field=bad_value)` raises immediately

Examples:

```
from pydantic import BaseModel, ValidationError

class Item(BaseModel):
    name: str
    price: float
```

```

    quantity: int

# Coercion: "3" becomes 3
item = Item(name="Book", price=12.99, quantity="3")
print(item.quantity)  # 3

# ValidationError: "free" cannot become float
try:
    Item(name="Book", price="free", quantity=1)
except ValidationError as e:
    print(e.errors())

```

Anti-patterns woven in:

- Catching `ValidationError` and silently ignoring it — this hides bugs; always log or re-raise with a meaningful response

01.2 Building Pydantic Models

Learning Objectives:

- Define a Pydantic model with required fields, optional fields, and default values
- Use common field types: `str`, `int`, `float`, `bool`, `list`, `datetime`
- Nest one model inside another to represent complex data shapes

Content Outline:

- Defining a model: subclass `BaseModel`, annotate fields with types
- Required fields vs optional fields: `Optional[str]` and `field: str = "default"`
- Common field types and what Pydantic does with each
- Nested models: a `Product` model that contains an `Address` model — Pydantic validates the nested object recursively
- Accessing model data: attribute access, `.model_dump()` to get a plain dict, `.model_dump_json()` for JSON

Transitions: You can now define data schemas in isolation. The next lesson gives you hands-on practice before we apply these models inside FastAPI endpoints.

Key Principles:

- A field without a default is required — omitting it raises `ValidationError`
- `Optional[str]` means the field can be `None`, not that it can be omitted — use `= None` to make it truly optional
- `.model_dump()` converts a model instance back to a plain dict — essential for passing data to ORMs or serializing to JSON

Examples:

```

from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class Address(BaseModel):
    street: str
    city: str
    zip_code: str

class User(BaseModel):
    id: int
    name: str
    email: str
    bio: Optional[str] = None
    created_at: datetime = datetime.now()
    address: Address

user = User(
    id=1,
    name="Ana",
    email="ana@example.com",
    address={"street": "123 Main St", "city": "Madrid", "zip_code": "28001"}
)
print(user.model_dump())

```

Anti-patterns woven in:

- `Optional[str]` without `= None` — the field still requires an explicit `None` to be passed; beginners often expect it to be omissible
- Using a plain `dict` for nested objects when you have a model — you lose validation on the nested data

01.3 Define a Schema

Challenge Prompt: Define a `Product` Pydantic model with the fields `name` (str), `price` (float), `stock` (int, default 0), and an optional `description` (str). Instantiate it with valid data and print the result as a dictionary.

Solution Code:

```

from pydantic import BaseModel
from typing import Optional

class Product(BaseModel):
    name: str
    price: float
    stock: int = 0
    description: Optional[str] = None

```

```
product = Product(name="Laptop", price=999.99, stock=5)
print(product.model_dump())
```

Challenge Code:

```
from pydantic import BaseModel
from typing import Optional

class Product(BaseModel):
    # your code here: define name, price, stock, and description fields

product = Product(name="Laptop", price=999.99, stock=5)
print(product.model_dump())
```

02.0 Pydantic in FastAPI

Learning Objectives:

- Understand the two points where Pydantic integrates with FastAPI: request bodies and response models
- Describe the data flow: JSON in → Pydantic model → Python object → endpoint logic → Python object → Pydantic model → JSON out
- Preview what the sub-lessons cover: validating incoming data and shaping outgoing data

Content Outline:

- FastAPI's native Pydantic integration — no extra setup needed
- The two integration points: request body declaration and `response_model` parameter
- What happens automatically: JSON parsing, type validation, 422 error on bad input, serialization on output
- Why this matters: your endpoint function receives a clean, typed Python object — no manual parsing

Transitions: Lesson 02.1 focuses on the first integration point: using a Pydantic model to validate data coming into an endpoint.

Key Principles:

- FastAPI uses Pydantic under the hood — every model you define becomes part of the auto-generated OpenAPI docs
- The same model can be used for both input and output, or you can define separate models for each direction — each approach has trade-offs

Examples:

- A FastAPI endpoint that declares `body: UserCreate` as a parameter — FastAPI reads the JSON body, validates it against `UserCreate`, and passes a typed `UserCreate` instance to the function
 - The same endpoint with `response_model=UserPublic` — FastAPI serializes the return value through `UserPublic` before sending the response
-

02.1 Request Body Validation

Learning Objectives:

- Declare a Pydantic model as a request body parameter in a FastAPI endpoint
- Explain what FastAPI does automatically: parse JSON, validate fields, return 422 on failure
- Access validated fields from the model instance inside the endpoint function

Content Outline:

- Declaring a request body: `def create_user(user: UserCreate):`
- What FastAPI does: reads the JSON body, instantiates `UserCreate`, raises HTTP 422 if validation fails
- Accessing data: `user.name`, `user.email` — plain attribute access on the validated model
- The 422 Unprocessable Entity response: what it looks like, what information it includes
- When to use a request body vs query parameters — bodies for structured data, params for filters/pagination

Transitions: Now that incoming data is validated, lesson 02.2 covers the other direction: controlling what your endpoint sends back with a response model.

Key Principles:

- FastAPI injects the validated model instance directly into your function — you never touch raw JSON
- A 422 response is returned automatically with a structured error body — you do not need to write that error handling yourself
- Pydantic validation runs before your endpoint logic executes — by the time your code runs, the data is guaranteed valid

Examples:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: str
    age: int

@app.post("/users")
def create_user(user: UserCreate):
```

```
# user is already validated – access fields directly
return {"message": f"Created user {user.name}", "email": user.email}
```

Anti-patterns woven in:

- Declaring the body as `body: dict` and parsing manually inside the function — you lose validation, lose IDE support, and write boilerplate that Pydantic eliminates
 - Catching `ValidationError` inside the endpoint — FastAPI handles this automatically; catching it manually usually means fighting the framework
-

02.2 Response Model Serialization 📖

Learning Objectives:

- Use `response_model` in a FastAPI route decorator to define the response schema
- Explain how FastAPI filters output fields using the response model
- Define separate input and output models to control what data is exposed to clients

Content Outline:

- `response_model`: passed to the route decorator, not the function signature
- What it does: FastAPI takes the return value, passes it through the response model, and serializes to JSON — extra fields are stripped, missing fields raise an error
- The input/output model pattern: `UserCreate` for incoming data (includes password), `UserPublic` for outgoing data (excludes password)
- `response_model_exclude_unset`: only include fields that were explicitly set — useful for partial responses

Transitions: The next lesson puts both pieces together: you will build an endpoint that validates incoming data with one model and serializes outgoing data with another.

Key Principles:

- `response_model` is a filter, not just documentation — it actively strips fields not in the model before sending the response
- Never return a model that includes sensitive fields (passwords, tokens) — define a separate public model
- FastAPI validates the return value against `response_model` in development — if your endpoint returns unexpected data, you find out immediately

Examples:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
```



```

    name: str
    email: str
    password: str

class UserPublic(BaseModel):
    name: str
    email: str

@app.post("/users", response_model=UserPublic)
def create_user(user: UserCreate):
    # password is in user but will NOT appear in the response
    return user # FastAPI filters through UserPublic

```

Anti-patterns woven in:

- Returning the full ORM object or the full input model without a response model — sensitive fields leak to the client; always define what you expose explicitly
- Using `dict` as the return type and manually popping sensitive keys — fragile, easy to forget a new field; `response_model` handles this declaratively

02.3 Build a Complete Endpoint 🖥️

Challenge Prompt: Build a FastAPI POST endpoint at `/products` that accepts a `ProductCreate` body (fields: `name`, `price`, `stock`) and returns only `name` and `price` using a `ProductPublic` response model.

Solution Code:

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ProductCreate(BaseModel):
    name: str
    price: float
    stock: int

class ProductPublic(BaseModel):
    name: str
    price: float

@app.post("/products", response_model=ProductPublic)
def create_product(product: ProductCreate):
    return product

```

Challenge Code:

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Define ProductCreate with name, price, stock
# Define ProductPublic with name, price only

# your code here: create the POST /products endpoint

```

03.0 Assessment 🧐

Learning Objectives:

- Demonstrate understanding of Pydantic's validation and serialization mechanics
- Identify correct and incorrect uses of request body declaration and response models in FastAPI
- Distinguish between required, optional, and default fields in a Pydantic model

Question Format: Scenario-based multiple choice

Questions:

1. You define `class Item(BaseModel): price: float`. A client sends `{"price": "19.99"}`. What does Pydantic do?

- a) Raises a `ValidationError` because `"19.99"` is a string
- b) Coerces `"19.99"` to `19.99` and creates the model ✓
- c) Stores `"19.99"` as a string without error
- d) Returns `None` for the `price` field

2. You have `class User(BaseModel): name: str; bio: Optional[str]`. What happens when you call `User(name="Ana")`?

- a) Raises `ValidationError` because `bio` is missing
- b) Creates the model with `bio` set to `None` ✓ ← wait, this is wrong. `Optional[str]` without `= None` still requires explicit `None` or the field... actually in Pydantic v2, `Optional[str]` without a default IS still required. Let me reconsider.

Actually, in Pydantic v2, `bio: Optional[str]` without a default value means the field is required but can be `None`. To make it truly optional (omissible), you need `bio: Optional[str] = None`. This is an important gotcha that we taught in 01.2.

Let me revise the question:

2. You have `class User(BaseModel): name: str; bio: Optional[str]` (no default). What happens when you call `User(name="Ana")`?

- a) Creates the model with `bio` set to `None` automatically

- b) Raises `ValidationError` because `bio` is required — `Optional` does not mean omissible ✓
 - c) Creates the model and ignores the missing `bio`
 - d) Raises a `TypeError` from Python, not Pydantic
3. Your FastAPI endpoint declares `def get_user(user: UserResponse)`. A client sends a POST request with a missing required field. What happens?
- a) The endpoint runs and raises a `KeyError` inside your function
 - b) FastAPI returns a 500 Internal Server Error
 - c) FastAPI returns a 422 Unprocessable Entity with details about the missing field ✓
 - d) The missing field is set to `None` automatically
4. You want to accept a `password` field on input but never expose it in the response. Which approach is correct?
- a) Accept `UserCreate` (with `password`) and manually `del` the password before returning
 - b) Use `response_model=UserPublic` where `UserPublic` has no `password` field ✓
 - c) Declare the endpoint return type as `dict` and pop the key
 - d) Use `Optional[str]` on the password field in the response model
5. You call `.model_dump()` on a Pydantic model instance. What do you get?
- a) A JSON string
 - b) A plain Python dictionary with the model's field values ✓
 - c) A new Pydantic model with the same values
 - d) A list of (field, value) tuples
6. Which of the following correctly declares a FastAPI endpoint that validates a JSON request body?
- a) `def create(body: dict):` — raw dict, no validation
 - b) `def create(body: ItemCreate):` where `ItemCreate` is a `BaseModel` subclass ✓
 - c) `def create(body = Body(...)):` with no model
 - d) `def create(body: str):` and parse JSON manually
7. A `ProductCreate` model has `stock: int = 0`. A client sends `{"name": "Book", "price": 9.99}` without the `stock` field. What happens?
- a) Raises `ValidationError` — `stock` is required
 - b) Sets `stock` to `None`
 - c) Creates the model with `stock = 0` ✓
 - d) Raises a `KeyError` when you access `product.stock`
8. You return a `UserCreate` instance from an endpoint with `response_model=UserPublic`. `UserPublic` has only `name` and `email`. The `UserCreate` instance also has `password`. What does the client receive?
- a) All fields including `password`
 - b) Only `name` and `email` — `response_model` filters out fields not in `UserPublic` ✓
 - c) A serialization error because `UserCreate` and `UserPublic` are different types

- d) An empty response

Evaluation Criteria:

- 8/8: Excellent mastery of Pydantic validation, model definition, and FastAPI integration
- 6–7/8: Strong understanding with minor gaps — review the lessons on coercion or response models
- 4–5/8: Core concepts are there but some mechanics are unclear — revisit sections 01 and 02
- Below 4: Revisit the full course before continuing

Score Interpretation: Questions 1–2 test Section 01 (Pydantic mechanics and model definition). Questions 3–4 and 6 test Section 02 (FastAPI integration). Questions 5, 7, and 8 test applied understanding across both sections.

04.0 Wrapping Up

Content Outline:

- Course recap: what students accomplished
 - Learned what Pydantic is and how it validates data using type annotations
 - Built Pydantic models with required fields, optional fields, defaults, and nested models
 - Used Pydantic models as FastAPI request bodies for automatic input validation
 - Defined response models to control serialization and protect sensitive data
- Connection to bigger picture: Pydantic is used throughout production FastAPI applications — in database schemas (with SQLAlchemy), settings management, and anywhere data crosses a boundary
- Next steps: explore Pydantic validators (`@field_validator`), `model_config`, and integration with ORMs using `from_orm`
- Resources for further exploration:
 - Pydantic v2 documentation: <https://docs.pydantic.dev>
 - FastAPI request body docs: <https://fastapi.tiangolo.com/tutorial/body/>
 - FastAPI response model docs: <https://fastapi.tiangolo.com/tutorial/response-model/>
- Encouragement: You now have one of the most practically useful skills in FastAPI development. Every real-world API you build from here will use what you learned in this course.

Note: This is conclusion only — no theory, exercises, or assessment.